

main.c



Share

Run

Output

```
1 #include <stdio.h>
2 #include <string.h>
3 int main() {
4     char user[10][20], file[10][20][20];
5     int users = 0, files[10] = {0};
6     int choice, u, i;
7     char uname[20], fname[20];
8     while (1) {
9         printf("\n--- Two Level Directory ---\n");
10        printf("1. Create User\n");
11        printf("2. Create File\n");
12        printf("3. Display Files\n");
13        printf("4. Search File\n");
14        printf("5. Exit\n");
15        printf("Enter choice: ");
16        scanf("%d", &choice);
17        if (choice == 1) {
18            printf("Enter user name: ");
19            scanf("%s", uname);
20            strcpy(user[users], uname);
21            users++;
22            printf("User created successfully.\n");
```

```
--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 1
Enter user name: user1
User created successfully.

--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 2
Enter user name: user1
Enter file name: file1
File created successfully.
```

main.c

Share

Run

```
3/      scanf("%s", name);
38+    for (i = 0; i < n; i++) {
39+        if (strcmp(files[i], name) == 0) {
40            printf("File found.\n");
41            break;
42        }
43    }
44    if (i == n)
45        printf("File not found.\n");
46 }
47+ else if (choice == 4) {
48     printf("\nFiles in Directory:\n");
49     for (i = 0; i < n; i++)
50         printf("%s\n", files[i]);
51 }
52 else if (choice == 5)
53     break;
54 else
55     printf("Invalid choice.\n");
56 }
57 return 0;
58 }
```

Output

Enter choice: 1
Enter file name: file2
File created successfully.

--- Single Level Directory ---
1. Create File
2. Delete File
3. Search File
4. Display Files
5. Exit
Enter choice: 4

Files in Directory:
file1
file2

--- Single Level Directory ---
1. Create File
2. Delete File
3. Search File
4. Display Files
5. Exit

main.c

```
55- if (u == users) {
56-     printf("User not found.\n");
57- } else {
58-     printf("Enter file name: ");
59-     scanf("%s", fname);
60-     for (i = 0; i < files[u]; i++) {
61-         if (strcmp(file[u][i], fname) == 0)
62-             break;
63-     }
64-     if (i < files[u])
65-         printf("File found.\n");
66-     else
67-         printf("File not found.\n");
68- }
69- }
70- else if (choice == 5)
71-     break;
72- else
73-     printf("Invalid choice.\n");
74- }
75- return 0;
76- }
```



Share

Run

Output

```
--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 1
Enter user name: user1
User created successfully.

--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 2
Enter user name: user1
Enter file name: file1
File created successfully.
```

main.c



Share

Run

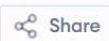
Output

```
23     }
24     else if (choice == 2) {
25         printf("Enter user name: ");
26         scanf("%s", uname);
27         for (u = 0; u < users; u++) {
28             if (strcmp(user[u], uname) == 0)
29                 break;
30         }
31         if (u == users) {
32             printf("User not found.\n");
33         } else {
34             printf("Enter file name: ");
35             scanf("%s", fname);
36             strcpy(file[u][files[u]], fname);
37             files[u]++;
38             printf("File created successfully.\n");
39         }
40     }
41     else if (choice == 3) {
42         for (u = 0; u < users; u++) {
43             printf("\nUser: %s\n", user[u]);
44             for (i = 0; i < files[u]; i++)
```

```
--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 1
Enter user name: user1
User created successfully.

--- Two Level Directory ---
1. Create User
2. Create File
3. Display Files
4. Search File
5. Exit
Enter choice: 2
Enter user name: user1
Enter file name: file1
File created successfully.
```

main.c



Run

Output

```
1 #include <stdio.h>
2 #include <string.h>
3 int main() {
4     char files[20][20];
5     int n = 0, choice, i;
6     char name[20];
7     while (1) {
8         printf("\n--- Single Level Directory ---\n");
9         printf("1. Create File\n");
10        printf("2. Delete File\n");
11        printf("3. Search File\n");
12        printf("4. Display Files\n");
13        printf("5. Exit\n");
14        printf("Enter choice: ");
15        scanf("%d", &choice);
16        if (choice == 1) {
17            printf("Enter file name: ");
18            scanf("%s", name);
19            strcpy(files[n], name);
20            n++;
21            printf("File created successfully.\n");
22        }
```

```
--- Single Level Directory ---
1. Create File
2. Delete File
3. Search File
4. Display Files
5. Exit
Enter choice: 1
Enter file name: file1
File created successfully.

--- Single Level Directory ---
1. Create File
2. Delete File
3. Search File
4. Display Files
5. Exit
Enter choice: 1
Enter file name: file2
File created successfully.

--- Single Level Directory ---
```

main.c

```
30         else if(choice == 2)
31         {
32             if(pos == -1 || block[j] < block[pos])
33                 pos = j;
34         }
35         else if(choice == 3)
36         {
37             if(pos == -1 || block[j] > block[pos])
38                 pos = j;
39         }
40     }
41 }
42 if(pos != -1)
43 {
44     printf("Process %d allocated to Block %d\n", i + 1, pos + 1);
45     block[pos] -= process[i];
46 }
47 else
48     printf("Process %d cannot be allocated\n", i + 1);
49 }
50 return 0;
51 }
```

Share

Run

Output

Enter number of memory blocks: 5
Enter block sizes:
100 500 200 300 600
Enter number of processes: 4
Enter process sizes:
212 417 112 426

1. First Fit
2. Best Fit
3. Worst Fit

Enter choice: 1

Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2
Process 4 cannot be allocated

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1 #include <stdio.h>
2- struct Employee {
3     int id;
4     char name[20];
5     float salary;
6 };
7- int main() {
8     FILE *fp;
9     struct Employee e;
10    int n, pos;
11    fp = fopen("employee.dat", "w+");
12    printf("Enter number of employees: ");
13    scanf("%d", &n);
14-    for (int i = 0; i < n; i++) {
15        printf("\nEnter Employee ID: ");
16        scanf("%d", &e.id);
17        printf("Enter Name: ");
18        scanf("%s", e.name);
19        printf("Enter Salary: ");
20        scanf("%f", &e.salary);
21        fwrite(&e, sizeof(e), 1, fp);
22    }
```

Enter number of employees: 3

Enter Employee ID: 101

Enter Name: arun

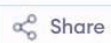
Enter Salary: 25000

Segmentation fault

=== Code Exited With Errors ===

main.c		Output
23-	<pre>else if (choice == 2) {</pre>	<pre>--- Single Level Directory --- 1. Create File 2. Delete File 3. Search File 4. Display Files 5. Exit Enter choice: 1 Enter file name: file1 File created successfully.</pre>
24	<pre> printf("Enter file name to delete: ");</pre>	
25	<pre> scanf("%s", name);</pre>	
26-	<pre> for (i = 0; i < n; i++) {</pre>	
27-	<pre> if (strcmp(files[i], name) == 0) {</pre>	
28	<pre> strcpy(files[i], files[n - 1]);</pre>	
29	<pre> n--;</pre>	
30	<pre> printf("File deleted successfully.\n");</pre>	
31	<pre> break;</pre>	
32	<pre> }</pre>	
33	<pre> }</pre>	<pre>--- Single Level Directory --- 1. Create File 2. Delete File 3. Search File 4. Display Files 5. Exit Enter choice: 1 Enter file name: file2 File created successfully.</pre>
34	<pre>}</pre>	
35-	<pre>else if (choice == 3) {</pre>	
36	<pre> printf("Enter file name to search: ");</pre>	
37	<pre> scanf("%s", name);</pre>	
38-	<pre> for (i = 0; i < n; i++) {</pre>	
39-	<pre> if (strcmp(files[i], name) == 0) {</pre>	
40	<pre> printf("File found.\n");</pre>	
41	<pre> break;</pre>	
42	<pre> }</pre>	
43	<pre> }</pre>	<pre>--- Single Level Directory ---</pre>
44	<pre> if (i == n)</pre>	

main.c



Run

Output

```
12 printf("Enter number of employees: ");
13 scanf("%d", &n);
14 for (int i = 0; i < n; i++) {
15     printf("\nEnter Employee ID: ");
16     scanf("%d", &e.id);
17     printf("Enter Name: ");
18     scanf("%s", e.name);
19     printf("Enter Salary: ");
20     scanf("%f", &e.salary);
21     fwrite(&e, sizeof(e), 1, fp);
22 }
23 printf("\nEnter employee record number to view: ");
24 scanf("%d", &pos);
25 fseek(fp, (pos - 1) * sizeof(e), SEEK_SET);
26 fread(&e, sizeof(e), 1, fp);
27 printf("\nEmployee Details\n");
28 printf("ID      : %d\n", e.id);
29 printf("Name    : %s\n", e.name);
30 printf("Salary  : %.2f\n", e.salary);
31 fclose(fp);
32 return 0;
33 }
```

Enter number of employees: 3

Enter Employee ID: 101

Enter Name: arun

Enter Salary: 25000

Segmentation fault

=== Code Exited With Errors ===

main.c

Share

Run

```
1 #include <stdio.h>
2 int main()
3 {
4     int block[10], process[10], n, m, i, j, choice;
5     printf("Enter number of memory blocks: ");
6     scanf("%d", &n);
7     printf("Enter block sizes:\n");
8     for(i = 0; i < n; i++)
9         scanf("%d", &block[i]);
10    printf("Enter number of processes: ");
11    scanf("%d", &m);
12    printf("Enter process sizes:\n");
13    for(i = 0; i < m; i++)
14        scanf("%d", &process[i]);
15    printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\n");
16    printf("Enter choice: ");
17    scanf("%d", &choice);
18    for(i = 0; i < m; i++)
19    {
20        int pos = -1;
21        for(j = 0; j < n; j++)
22            {
```

Output

Enter number of memory blocks: 5
Enter block sizes:
100 500 200 300 600
Enter number of processes: 4
Enter process sizes:
212 417 112 426

1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 1
Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2
Process 4 cannot be allocated

=== Code Execution Successful ===

main.c



Run

Output

```
9 sem_wait(&forks[1]);
10 sem_wait(&forks[(i + 1) % 5]);
11 printf("Philosopher %d is eating\n", i + 1);
12 sem_post(&forks[i]);
13 sem_post(&forks[(i + 1) % 5]);
14 return NULL;
15 }
16 int main()
17 {
18     pthread_t p[5];
19     int i, id[5];
20     for(i = 0; i < 5; i++)
21         sem_init(&forks[i], 0, 1);
22     for(i = 0; i < 5; i++)
23     {
24         id[i] = i;
25         pthread_create(&p[i], NULL, eat, &id[i]);
26     }
27     for(i = 0; i < 5; i++)
28         pthread_join(p[i], NULL);
29     return 0;
```

Philosopher 1 is thinking
Philosopher 1 is eating
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 3 is eating
Philosopher 5 is thinking
Philosopher 5 is eating
Philosopher 2 is eating
Philosopher 4 is thinking
Philosopher 4 is eating

=== Code Execution Successful ===

main.c



Run

Output

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 sem_t forks[5];
5 void *eat(void *arg)
6 {
7     int i = *(int *)arg;
8     printf("Philosopher %d is thinking\n", i + 1);
9     sem_wait(&forks[i]);
10    sem_wait(&forks[(i + 1) % 5]);
11    printf("Philosopher %d is eating\n", i + 1);
12    sem_post(&forks[i]);
13    sem_post(&forks[(i + 1) % 5]);
14    return NULL;
15 }
16 int main()
17 {
18     pthread_t p[5];
19     int i, id[5];
20     for(i = 0; i < 5; i++)
21         sem_init(&forks[i], 0, 1);
22     for(i = 0; i < 5; i++)
```

Philosopher 1 is thinking
Philosopher 1 is eating
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 3 is eating
Philosopher 5 is thinking
Philosopher 5 is eating
Philosopher 2 is eating
Philosopher 4 is thinking
Philosopher 4 is eating

=== Code Execution Successful ===

main.c				 Share	Run	Output
<pre>1 #include <stdio.h> 2 #include <pthread.h> 3 void *task(void *arg) 4 { 5 printf("Thread is running\n"); 6 return NULL; 7 } 8 9 int main() 10 { 11 pthread_t t1, t2; 12 13 pthread_create(&t1, NULL, task, NULL); 14 pthread_create(&t2, NULL, task, NULL); 15 16 pthread_join(t1, NULL); 17 pthread_join(t2, NULL); 18 19 printf("Main thread completed\n"); 20 return 0; 21 }</pre>						Thread is running Thread is running Main thread completed === Code Execution Successful ===

main.c

```
1/-      for(i = 0; i < n; i++) {
18-          if(rt[i] > 0) {
19-              done = 0;
20-              if(rt[i] > tq) {
21-                  time += tq;
22-                  rt[i] -= tq;
23-              } else {
24-                  time += rt[i];
25-                  wt[i] = time - bt[i];
26-                  rt[i] = 0;
27-              }
28-          }
29-      }
30-  } while(done == 0);
31-  printf("\nProcess\tBurst\tWaiting\tTurnaround\n");
32-  for(i = 0; i < n; i++) {
33-      tat[i] = wt[i] + bt[i];
34-      printf("P%d\t%d\t%d\t%d\n",
35-          i + 1, bt[i], wt[i], tat[i]);
36-  }
37-  return 0;
38- }
```

Share

Run

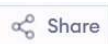
Output

Enter number of processes: 3
Enter burst time for P1: 5
Enter burst time for P2: 3
Enter burst time for P3: 4
Enter time quantum: 2

Process	Burst	Waiting	Turnaround
P1	5	7	12
P2	3	6	9
P3	4	7	11

=== Code Execution Successful ===

main.c



Run

Output

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/msg.h>
4 #include <string.h>
5- struct msg {
6     long type;
7     char text[100];
8 };
9- int main() {
10     struct msg m;
11     int id;
12     id = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
13     m.type = 1;
14     strcpy(m.text, "Hello from Process 1");
15     msgsnd(id, &m, sizeof(m.text), 0);
16     printf("Message sent: %s\n", m.text);
17     msgrcv(id, &m, sizeof(m.text), 1, 0);
18     printf("Message received: %s\n", m.text);
19     msgctl(id, IPC_RMID, NULL);
20     return 0;
21 }
```

Message sent: Hello from Process 1
Message received: Hello from Process 1

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1 #include <stdio.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <string.h>
5
6 int main() {
7     int shmid;
8     char *str;
9
10    shmid = shmget(1234, 1024, 0666 | IPC_CREAT);
11
12    str = (char *)shmat(shmid, NULL, 0);
13
14    strcpy(str, "Hello from Shared Memory!");
15
16    printf("Message: %s\n", str);
17
18    shmdt(str);
19    shmctl(shmid, IPC_RMID, NULL);
20
21    return 0;
22 }
```

Message: Hello from Shared Memory!

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1 #include <stdio.h>
2 int main() {
3     int n, i, j, temp;
4     int bt[10], p[10], wt[10], tat[10];
5     printf("Enter number of processes: ");
6     scanf("%d", &n);
7     for(i = 0; i < n; i++) {
8         p[i] = i + 1;
9         printf("Enter burst time for P%d: ", p[i]);
10        scanf("%d", &bt[i]);
11    }
12    for(i = 0; i < n - 1; i++) {
13        for(j = i + 1; j < n; j++) {
14            if(bt[i] > bt[j]) {
15                temp = bt[i];
16                bt[i] = bt[j];
17                bt[j] = temp;
18                temp = p[i];
19                p[i] = p[j];
20                p[j] = temp;
21            }
22        }
23    }
```

Enter number of processes: 4
Enter burst time for P1: 6
Enter burst time for P2: 2
Enter burst time for P3: 8
Enter burst time for P4: 3

Process	Burst	Waiting	Turnaround
P2	2	0	2
P4	3	2	5
P1	6	5	11
P3	8	11	19

=== Code Execution Successful ===

main.c



Share

Run

Output

```
1 #include <stdio.h>
2 int main() {
3     int n, i, tq, time = 0, done;
4     int bt[10], rt[10], wt[10], tat[10];
5     printf("Enter number of processes: ");
6     scanf("%d", &n);
7     for(i = 0; i < n; i++) {
8         printf("Enter burst time for P%d: ", i + 1);
9         scanf("%d", &bt[i]);
10        rt[i] = bt[i];
11        wt[i] = 0;
12    }
13    printf("Enter time quantum: ");
14    scanf("%d", &tq);
15    do {
16        done = 1;
17        for(i = 0; i < n; i++) {
18            if(rt[i] > 0) {
19                done = 0;
20                if(rt[i] > tq) {
21                    time += tq;
22                    rt[i] -= tq;
```

Enter number of processes: 3
Enter burst time for P1: 5
Enter burst time for P2: 3
Enter burst time for P3: 4
Enter time quantum: 2

Process	Burst	Waiting	Turnaround
P1	5	7	12
P2	3	6	9
P3	4	7	11

=== Code Execution Successful ===

main.c

Share

Run

12-
13-
14-
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

```
for(i = 0; i < n - 1; i++) {  
    for(j = i + 1; j < n; j++) {  
        if(bt[i] > bt[j]) {  
            temp = bt[i];  
            bt[i] = bt[j];  
            bt[j] = temp;  
            temp = p[i];  
            p[i] = p[j];  
            p[j] = temp;  
        }  
    }  
}  
wt[0] = 0;  
for(i = 1; i < n; i++)  
    wt[i] = wt[i - 1] + bt[i - 1];  
for(i = 0; i < n; i++)  
    tat[i] = wt[i] + bt[i];  
printf("\nProcess\tBurst\tWaiting\tTurnaround\n");  
for(i = 0; i < n; i++)  
    printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);  
return 0;  
}
```

Output

Enter number of processes: 4
Enter burst time for P1: 6
Enter burst time for P2: 2
Enter burst time for P3: 8
Enter burst time for P4: 3

Process Burst Waiting Turnaround
P2 2 0 2
P4 3 2 5
P1 6 5 11
P3 8 11 19

=== Code Execution Successful ===

main.c

Share

Run

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

```
for(i = 0; i < n; i++)
    if(at[i] <= time && rt[i] > 0 &&
        (x == -1 || pr[i] < pr[x]))
        x = i;
if(x == -1) {
    time++;
    continue;
}
rt[x]--;
time++;
if(rt[x] == 0) {
    done++;
    tat[x] = time - at[x];
    wt[x] = tat[x] - bt[x];
}
}
printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");
for(i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n",
        i + 1, at[i], bt[i], pr[i], wt[i], tat[i]);
return 0;
}
```

Output

Enter number of processes: 3

Enter Arrival Time, Burst Time, Priority for P1: 0 5 2

Enter Arrival Time, Burst Time, Priority for P2: 1 3 1

Enter Arrival Time, Burst Time, Priority for P3: 2 4 3

Process	AT	BT	Priority	WT	TAT
P1	0	5	2	3	8
P2	1	3	1	0	3
P3	2	4	3	6	10

=== Code Execution Successful ===